

Relatório 21 - Prática: Visão Computacional (III)

Igor Carvalho Marchi

1. Introdução

O Card 21 aborda conceitos de Deep Learning aplicados à visão computacional com PyTorch, incluindo tensores, redes neurais, CNNs, Data Augmentation e Autoencoders. Como prática, foi desenvolvido um Autoencoder convolucional com o CIFAR-10 para reconstruir imagens após a adição de ruído.

2. Desenvolvimento

O conteúdo começa pelos fundamentos de Deep Learning e pelo uso do PyTorch. Os tensores são apresentados como a estrutura usada para armazenar e manipular os dados que entram na rede neural. Com o PyTorch também é possível utilizar GPU para acelerar operações e o treinamento dos modelos quando esse recurso está disponível.

Na construção das redes neurais são trabalhados conceitos como camadas, funções de ativação, função de perda e otimizadores. Durante o treinamento, o modelo recebe os dados, gera uma saída, calcula o erro e ajusta seus pesos para tentar melhorar nas próximas épocas. Também são utilizados Dataset e DataLoader para organizar os dados e enviá-los para o modelo em lotes.

2.1 Deep Learning Fundamentals

A primeira parte do curso introduz o Deep Learning e o PyTorch. Deep Learning utiliza redes neurais com várias camadas para aprender padrões a partir dos dados. Dentro dessa parte, o PyTorch aparece como a biblioteca usada para criar os modelos, manipular os dados e realizar o treinamento.

Também são apresentados os tensores. Eles são a estrutura principal usada pelo PyTorch para representar os dados. Em imagens, por exemplo, um tensor pode guardar os canais de cor, a altura e a largura. Durante as aulas são feitas operações como criação, alteração de formato, soma, multiplicação e movimentação dos tensores para CPU ou GPU.

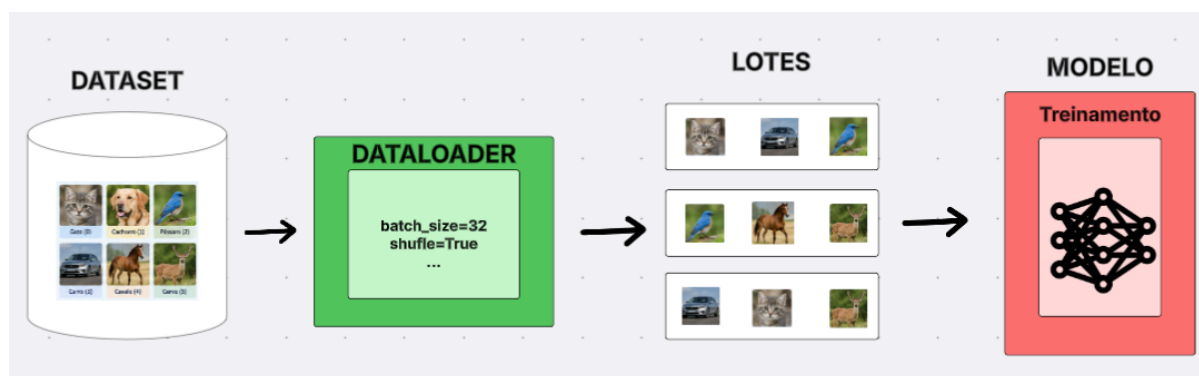
O uso de GPU é importante porque muitas operações de redes neurais envolvem grande quantidade de cálculos. Quando o ambiente possui CUDA disponível, o PyTorch consegue enviar os tensores e o modelo para a GPU, acelerando principalmente o treinamento.

2.2 Building Neural Networks with PyTorch

Depois dos fundamentos, o curso entra na construção de redes neurais. Uma rede é formada por camadas que recebem os dados, fazem transformações e produzem uma saída. Durante o treinamento, essa saída é comparada com o valor esperado para calcular o erro do modelo.

As funções de ativação aparecem nessa etapa para permitir que a rede aprenda relações mais complexas. Um exemplo bastante usado é a ReLU. Também são mostradas as funções de perda, que indicam o quanto a saída do modelo está diferente do resultado esperado, e os otimizadores, responsáveis por atualizar os pesos da rede para reduzir esse erro.

Outro conteúdo importante é o Dataset e o DataLoader. O Dataset representa os dados que serão usados, enquanto o DataLoader divide esses dados em lotes, chamados batches. Isso facilita o treinamento, porque o modelo recebe pequenas partes do conjunto de dados a cada passo. Também é mostrado o uso de Sequential, que permite organizar várias camadas em sequência.



2.3 Neural Networks for Images

Nessa parte o foco passa a ser imagens. O curso mostra como uma imagem é representada numericamente e como pode ser preparada antes de entrar em uma rede neural. Também é apresentada a classificação de imagens, em que o modelo recebe uma imagem e tenta identificar a qual classe ela pertence.

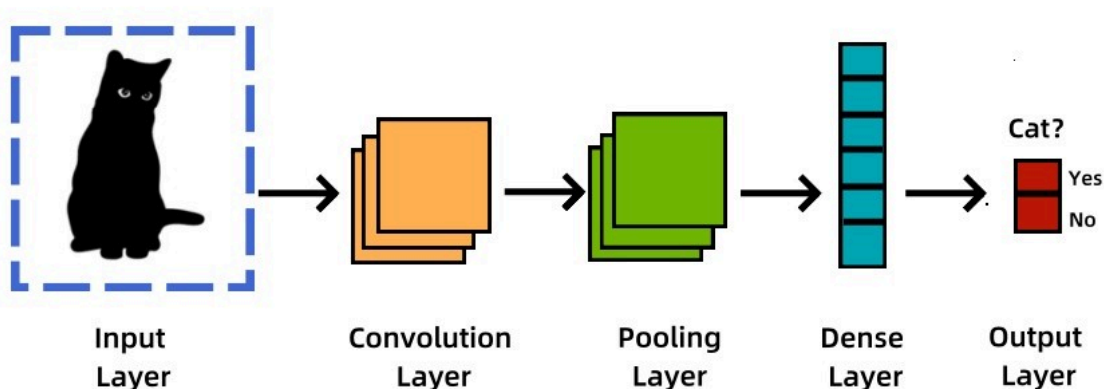
O ajuste de hiperparâmetros também aparece nessa seção. Elementos como taxa de aprendizado, quantidade de épocas e tamanho do batch podem mudar bastante o resultado de um treinamento. Por isso, esses valores precisam ser testados e ajustados de acordo com o problema.

A normalização dos dados é outro ponto importante, pois ajuda a deixar os valores de entrada em uma escala mais adequada. Isso pode deixar o treinamento mais estável e facilitar a convergência do modelo.

2.4 Convolutional Neural Networks (CNNs)

As Redes Neurais Convolucionais, ou CNNs, são apresentadas como um tipo de rede muito utilizado para imagens. Em vez de tratar todos os pixels de forma totalmente conectada desde o início, as CNNs usam filtros que percorrem a imagem e procuram características.

Nas primeiras camadas, esses filtros podem aprender padrões simples, como bordas, linhas e pequenas formas. Conforme a imagem passa por mais camadas, a rede consegue trabalhar com características mais complexas. As camadas de pooling podem ser usadas para reduzir as dimensões dos mapas de características e manter as informações mais relevantes.

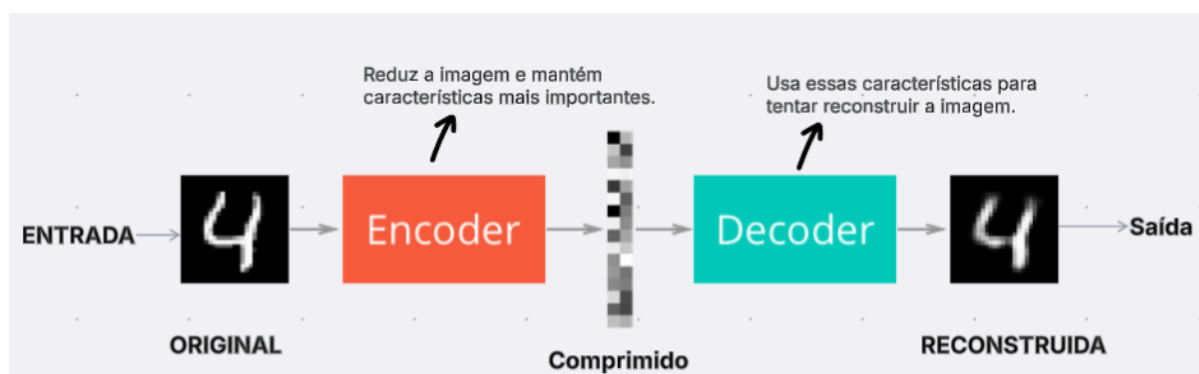


Também é estudado o Data Augmentation. A técnica cria variações das imagens de treinamento através de operações como cortes, espelhamentos ou pequenas alterações. A intenção é aumentar a variedade dos exemplos apresentados ao modelo sem precisar coletar um conjunto totalmente novo de imagens.

2.5 Auto Encoders

Os Autoencoders trabalham com uma ideia diferente da classificação. Em vez de gerar uma classe como saída, eles tentam reconstruir o próprio dado de entrada. A estrutura é dividida principalmente em Encoder e Decoder.

O Encoder recebe a entrada e reduz suas dimensões, criando uma representação menor que tenta guardar as informações mais importantes. O Decoder recebe essa representação e faz o caminho contrário, tentando reconstruir o dado original. Quando são usadas camadas convolucionais, esse processo pode ser aplicado diretamente a imagens.



O curso também apresenta o Variational Autoencoder, conhecido como VAE. Nesse caso, o espaço latente é tratado de forma probabilística, permitindo não apenas reconstruir os dados, mas também gerar novas amostras a partir do que foi aprendido.

2.6 Hands-on Projects

Na parte final são apresentados projetos práticos. O Neural Style Transfer combina o conteúdo de uma imagem com o estilo visual de outra. O Deep Fake mostra a manipulação de rostos utilizando redes neurais. Já o Image Colorization trabalha com a previsão de cores para imagens em preto e branco.

Esses projetos ajudam a mostrar que os mesmos conceitos vistos durante o curso podem ser usados para diferentes tarefas de visão computacional, não ficando limitados apenas à classificação de imagens.

3 Prática (Remoção de ruído utilizando Autoencoder)

Na prática foi utilizado o CIFAR-10 para treinar um Autoencoder capaz de receber imagens com ruído e tentar reconstruí-las. As imagens foram organizadas com o DataLoader, passaram pelo Encoder para redução das informações e depois pelo Decoder para reconstrução. O modelo foi treinado por 20 épocas e avaliado comparando as imagens originais com as reconstruídas.

4. Conclusão

O Card 21 ajudou a aprofundar os conhecimentos de Deep Learning e visão computacional utilizando PyTorch. Durante o conteúdo foram trabalhados conceitos como tensores, redes neurais, CNNs, Data Augmentation e Autoencoders. A parte prática ajudou a entender melhor como esses conceitos podem ser aplicados em imagens, principalmente no processo de treinamento e reconstrução usando Autoencoders.

REFERÊNCIA

Link da seção do card -

https://docs.google.com/document/d/1rAvJ0FiWZG5fgecj4VhHqDJcgQWh_O5CrmZCw812GGU/edit?tab=t.0#heading=h.xrjefw97or82